

Nombre del proyecto

D2. Gestor de pedidos para un emprendimiento local (catálogo, carrito, facturación simple).

Docente	Ing. Victoria Garcia
Asignatura	Diseño de Software
Estudiante	Jonnathan Gallegos
Fecha	26/07/2026

Resumen del Proyecto

Sistema web para la gestión de pedidos de una panadería, diseñado para automatizar el proceso de consulta de productos y registro de pedidos por parte de los clientes. La plataforma operará de manera autónoma, permitiendo la visualización del catálogo organizado por categorías y el almacenamiento de los pedidos. El sistema será desarrollado con PostgreSQL como base de datos, ASP.NET Core para el backend y Angular para el frontend.

FASE 1: Documento de diseño UML

Vista 4+1	Descripción
Vista lógica	Define las principales clases del sistema y sus relaciones, como usuarios, clientes, productos, categorías y pedidos.
Vista de desarrollo	Organiza la solución en un frontend desarrollado en Angular y un backend desarrollado en ASP.NET Core.
Vista de procesos	Describe el flujo de las operaciones principales, como el inicio de sesión, la consulta de productos y la realización de pedidos.
Vista física	Muestra el despliegue del sistema, donde la aplicación web se comunica con una API REST que almacena la información en PostgreSQL.
Vista de escenarios (+1)	Representa los principales casos de uso del sistema, como iniciar sesión, administrar productos, consultar productos y realizar pedidos.

Diagrama de casos de uso:

Sistema de Gestión de Pedidos - Diagrama de Casos de Uso

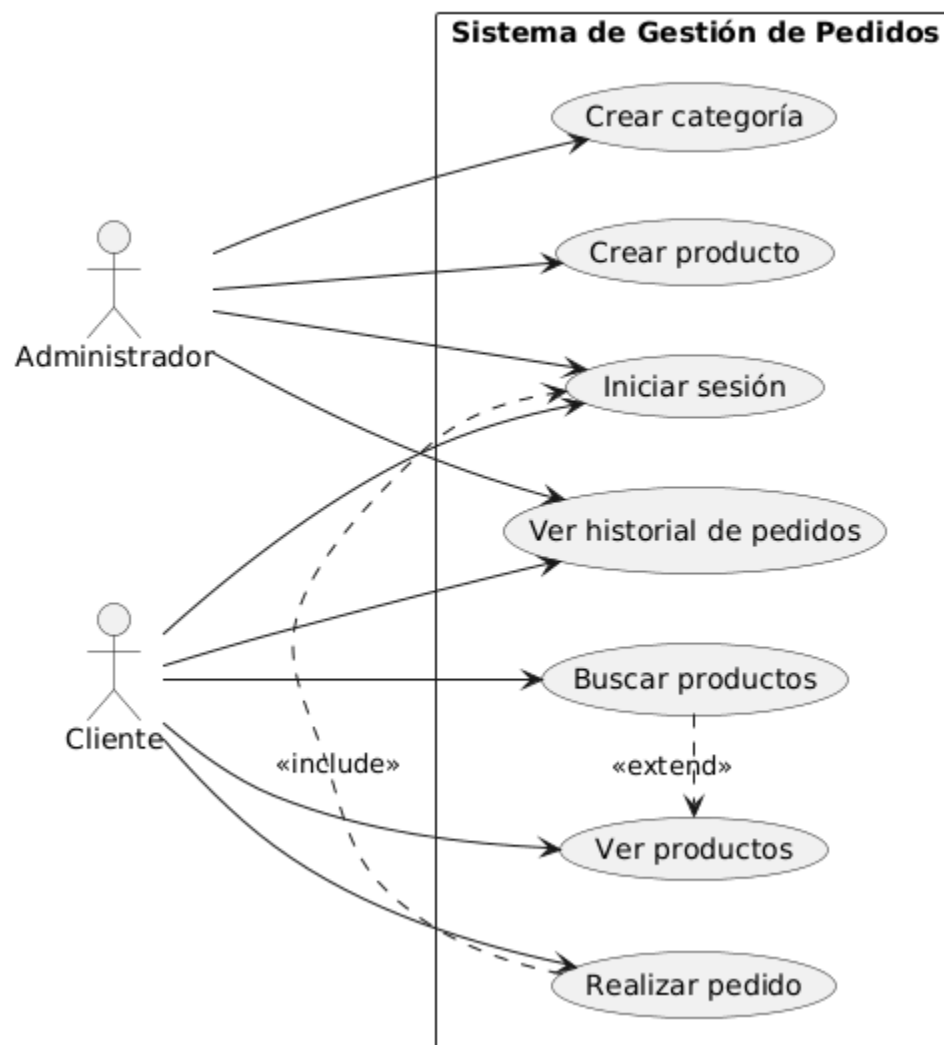


Diagrama de clases:

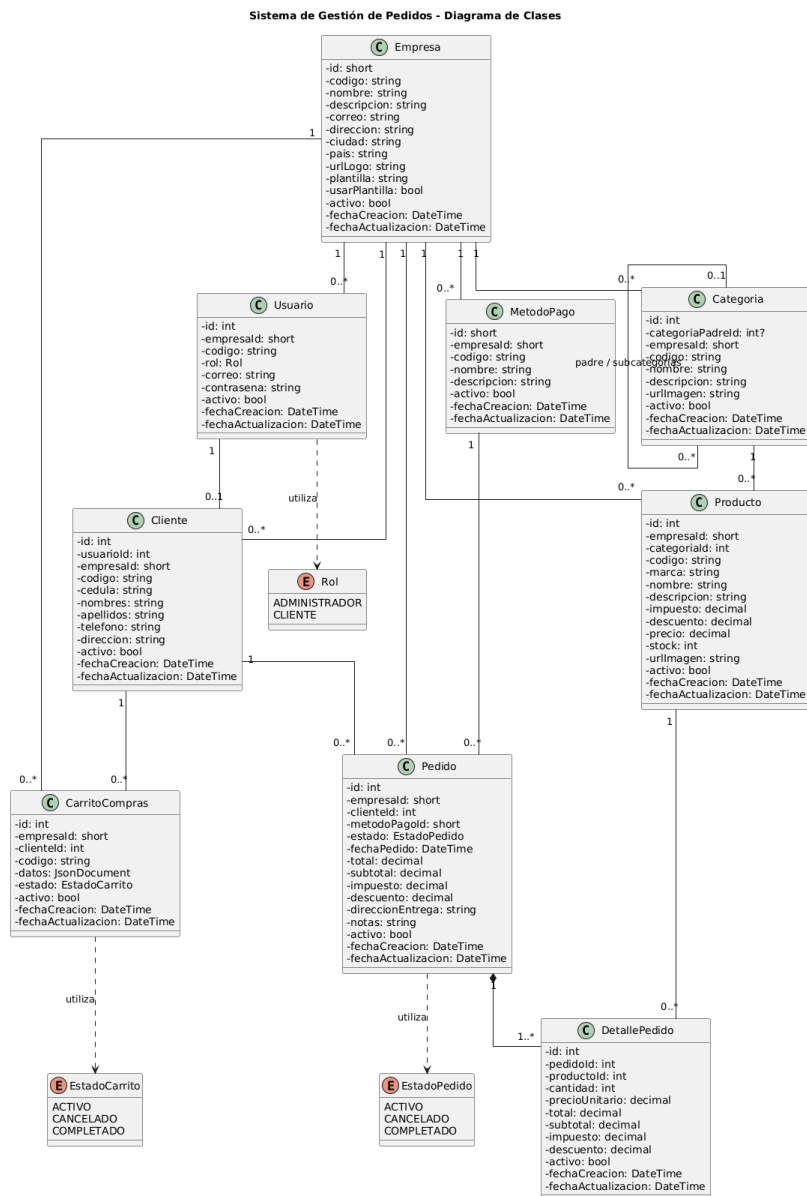
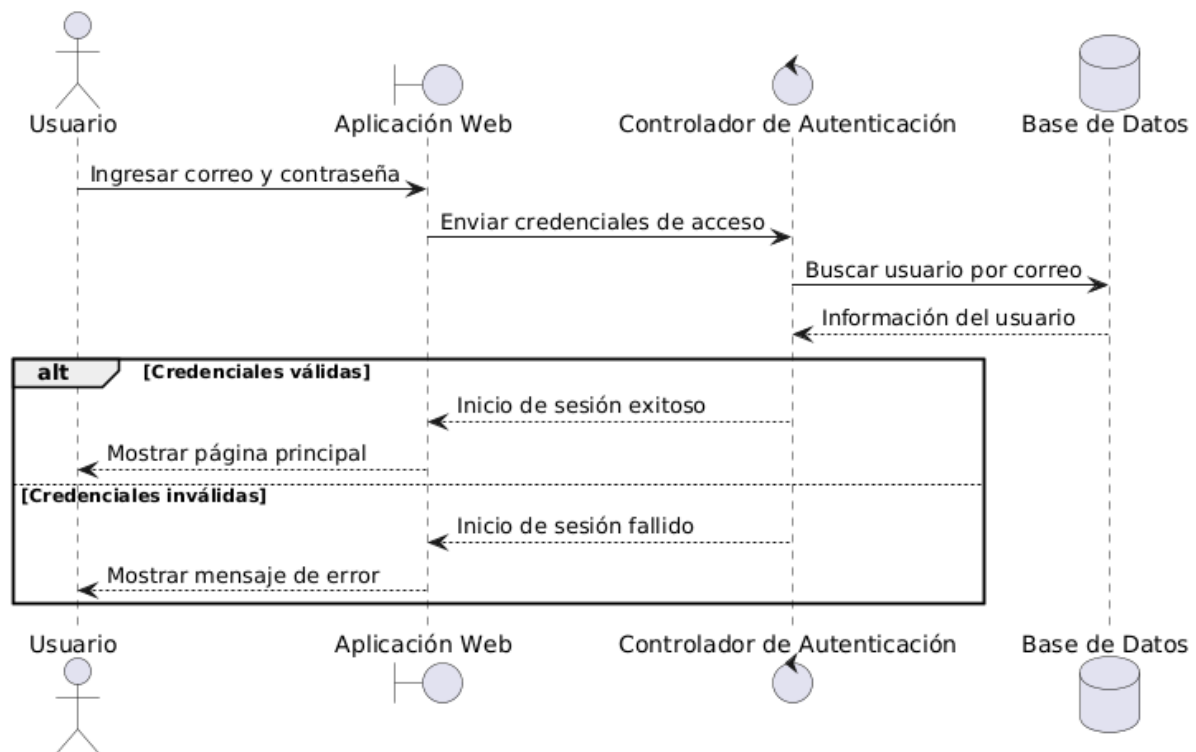


Diagrama de secuencia:

Inicio de sesión - Diagrama de Secuencia



Realizar pedido - Diagrama de Secuencia

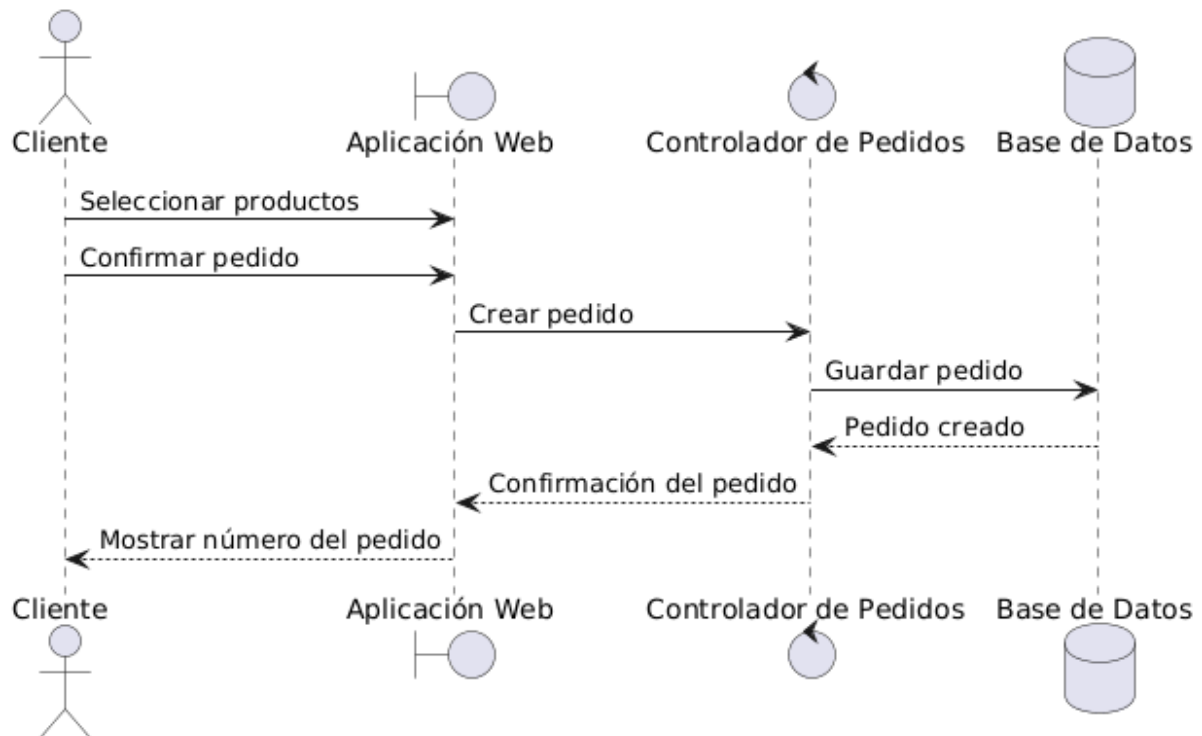
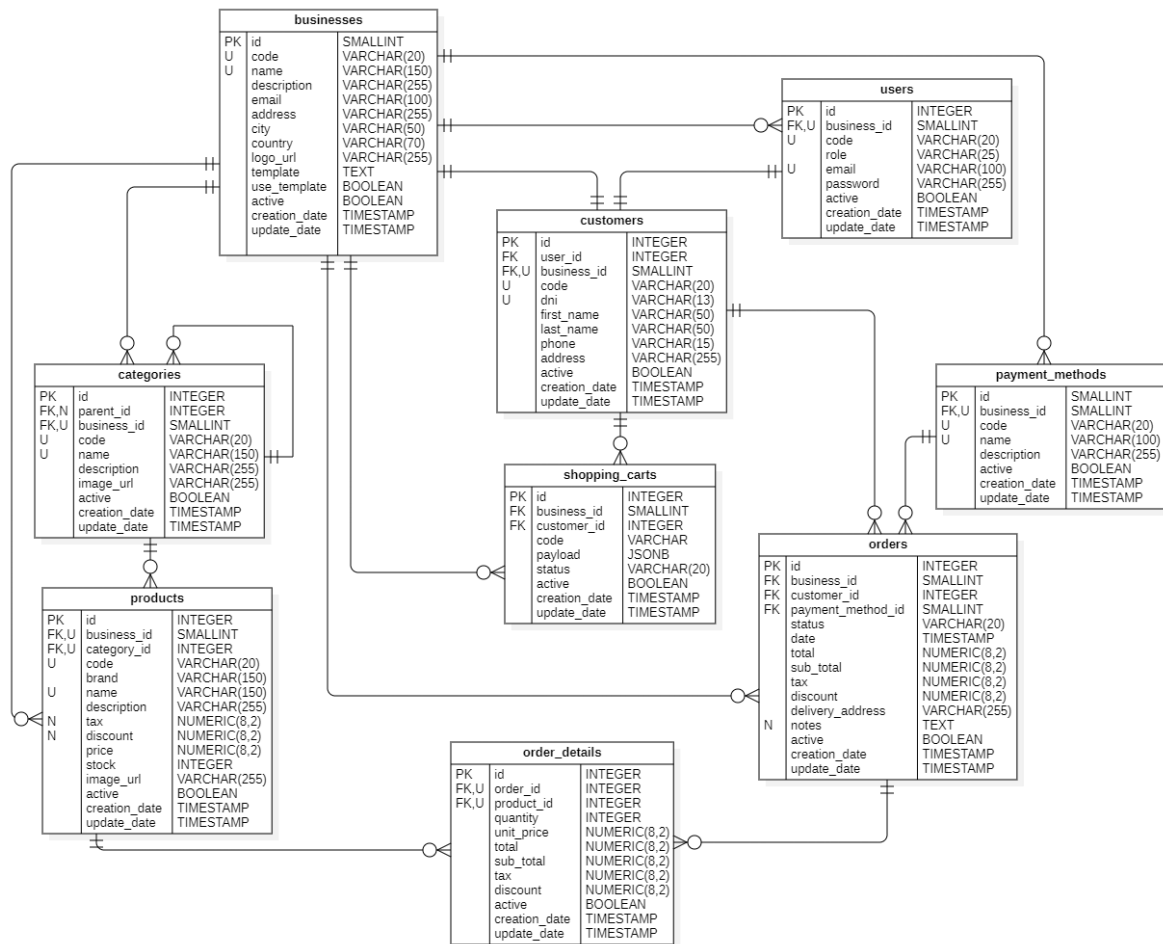


Diagrama ER:



DDL:

```

CREATE TABLE public.businesses (
    id smallint NOT NULL,
    code varchar(20) NOT NULL,
    name varchar(150) NOT NULL,
    description varchar(255) NOT NULL,
    email varchar(100) NOT NULL,
    address varchar(255) NOT NULL,
    city varchar(50) NOT NULL,
    country varchar(70) NOT NULL,
    logo_url varchar(255) NOT NULL,
    template text NOT NULL,
    use_template boolean NOT NULL,
    active boolean NOT NULL,
    creation_date timestamp with time zone NOT NULL,
    update_date timestamp with time zone NOT NULL,
    PRIMARY KEY (id)
);
    
```

```

ALTER TABLE public.businesses
ADD UNIQUE (code);
    
```

```

ALTER TABLE public.businesses
    
```

```

ADD UNIQUE (name);

ALTER TABLE public.businesses ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY
(CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 32767);

CREATE TABLE public.users (
id integer NOT NULL,
business_id smallint NOT NULL,
code varchar(20) NOT NULL,
role varchar(20) NOT NULL,
email varchar(100) NOT NULL,
password varchar(255) NOT NULL,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,
update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);

ALTER TABLE public.users
ADD UNIQUE (code);

ALTER TABLE public.users
ADD UNIQUE (business_id, email);

COMMENT ON COLUMN public.users.role
IS 'ADMIN
CUSTOMER';

ALTER TABLE public.users ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY
( CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647);

CREATE TABLE public.products (
id integer NOT NULL,
business_id smallint NOT NULL,
category_id integer NOT NULL,
code varchar(20) NOT NULL,
brand varchar(150) NOT NULL,
name varchar(150) NOT NULL,
description varchar(255) NOT NULL,
tax numeric(8,2),
discount numeric(8,2),
price numeric(8,2) NOT NULL,
stock integer NOT NULL,
image_url varchar(255) NOT NULL,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,
update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);
    
```

```

ALTER TABLE public.products
ADD UNIQUE (code);

ALTER TABLE public.products
ADD UNIQUE (business_id, category_id, name);

ALTER TABLE public.products ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY
( CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647);

CREATE TABLE public.customers (
id integer NOT NULL,
user_id integer NOT NULL,
business_id smallint NOT NULL,
code varchar(20) NOT NULL,
dni varchar(13) NOT NULL,
first_name varchar(50) NOT NULL,
last_name varchar(50) NOT NULL,
phone varchar(15) NOT NULL,
address varchar(255) NOT NULL,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,
update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);

ALTER TABLE public.customers
ADD UNIQUE (code);

ALTER TABLE public.customers
ADD UNIQUE (business_id, dni);

ALTER TABLE public.customers ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY
( CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647);

CREATE TABLE public.categories (
id integer NOT NULL,
parent_id integer,
business_id smallint NOT NULL,
code varchar(20) NOT NULL,
name varchar(150) NOT NULL,
description varchar(255) NOT NULL,
image_url varchar(255) NOT NULL,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,
update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);
    
```

```

ALTER TABLE public.categories
ADD UNIQUE (code);

ALTER TABLE public.categories
ADD UNIQUE (business_id, name);

CREATE INDEX ON public.categories
(parent_id);

ALTER TABLE public.categories ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY
( CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647);

CREATE TABLE public.order_details (
id integer NOT NULL,
order_id integer NOT NULL,
product_id integer NOT NULL,
quantity integer NOT NULL,
unit_price numeric(8,2) NOT NULL,
total numeric(8,2) NOT NULL,
sub_total numeric(8,2) NOT NULL,
tax numeric(8,2) NOT NULL,
discount numeric(8,2) NOT NULL,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,
update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);

ALTER TABLE public.order_details
ADD UNIQUE (order_id, product_id);

ALTER TABLE public.order_details ALTER COLUMN id ADD GENERATED BY DEFAULT AS
IDENTITY ( CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647);

CREATE TABLE public.orders (
id integer NOT NULL,
business_id smallint NOT NULL,
customer_id integer NOT NULL,
payment_method_id smallint NOT NULL,
status varchar(20) NOT NULL,
date timestamp with time zone NOT NULL,
total numeric(8,2) NOT NULL,
sub_total numeric(8,2) NOT NULL,
tax numeric(8,2) NOT NULL,
discount numeric(8,2) NOT NULL,
delivery_address varchar(255) NOT NULL,
notes text,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,

```

```

update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);

CREATE INDEX ON public.orders
(business_id);
CREATE INDEX ON public.orders
(customer_id);
CREATE INDEX ON public.orders
(payment_method_id);

COMMENT ON COLUMN public.orders.status
IS 'ACTIVE
CANCELLED
COMPLETED';

ALTER TABLE public.orders ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY
( CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647);

CREATE TABLE public.payment_methods (
id smallint NOT NULL,
business_id smallint NOT NULL,
code varchar(20) NOT NULL,
name varchar(100) NOT NULL,
description varchar(255) NOT NULL,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,
update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);

ALTER TABLE public.payment_methods
ADD UNIQUE (code);

ALTER TABLE public.payment_methods
ADD UNIQUE (business_id, name);

CREATE INDEX ON public.payment_methods
(business_id);

ALTER TABLE public.payment_methods ALTER COLUMN id ADD GENERATED BY DEFAULT AS
IDENTITY (CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 32767);

CREATE TABLE public.shopping_carts (
id integer NOT NULL,
business_id smallint NOT NULL,
customer_id integer NOT NULL,

```

```
code varchar NOT NULL,
payload JSONB NOT NULL,
status varchar(20) NOT NULL,
active boolean NOT NULL,
creation_date timestamp with time zone NOT NULL,
update_date timestamp with time zone NOT NULL,
PRIMARY KEY (id)
);

CREATE INDEX ON public.shopping_carts
(business_id);
CREATE INDEX ON public.shopping_carts
(customer_id);

COMMENT ON COLUMN public.shopping_carts.status
IS 'ACTIVE
CANCELLED
COMPLETED';

ALTER TABLE public.shopping_carts ALTER COLUMN id ADD GENERATED BY DEFAULT AS
IDENTITY ( CYCLE INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647);

ALTER TABLE public.users ADD CONSTRAINT FK_users__business_id FOREIGN KEY
(business_id) REFERENCES public.businesses(id);
ALTER TABLE public.products ADD CONSTRAINT FK_products__business_id FOREIGN KEY
(business_id) REFERENCES public.businesses(id);
ALTER TABLE public.products ADD CONSTRAINT FK_products__category_id FOREIGN KEY
(category_id) REFERENCES public.categories(id);
ALTER TABLE public.customers ADD CONSTRAINT FK_customers__user_id FOREIGN KEY
(user_id) REFERENCES public.users(id);
ALTER TABLE public.customers ADD CONSTRAINT FK_customers__business_id FOREIGN KEY
(business_id) REFERENCES public.businesses(id);
ALTER TABLE public.categories ADD CONSTRAINT FK_categories__parent_id FOREIGN KEY
(parent_id) REFERENCES public.categories(id);
ALTER TABLE public.categories ADD CONSTRAINT FK_categories__business_id FOREIGN
KEY (business_id) REFERENCES public.businesses(id);
ALTER TABLE public.order_details ADD CONSTRAINT FK_order_details__order_id FOREIGN
KEY (order_id) REFERENCES public.orders(id);
ALTER TABLE public.order_details ADD CONSTRAINT FK_order_details__product_id
FOREIGN KEY (product_id) REFERENCES public.products(id);
ALTER TABLE public.orders ADD CONSTRAINT FK_orders__business_id FOREIGN KEY
(business_id) REFERENCES public.businesses(id);
ALTER TABLE public.orders ADD CONSTRAINT FK_orders__customer_id FOREIGN KEY
(customer_id) REFERENCES public.customers(id);
ALTER TABLE public.orders ADD CONSTRAINT FK_orders__payment_method_id FOREIGN KEY
(payment_method_id) REFERENCES public.payment_methods(id);
ALTER TABLE public.payment_methods ADD CONSTRAINT FK_payment_methods__business_id
FOREIGN KEY (business_id) REFERENCES public.businesses(id);
```

```
ALTER TABLE public.shopping_carts ADD CONSTRAINT FK_shopping_carts__business_id
FOREIGN KEY (business_id) REFERENCES public.businesses(id);
ALTER TABLE public.shopping_carts ADD CONSTRAINT FK_shopping_carts__customer_id
FOREIGN KEY (customer_id) REFERENCES public.customers(id);
```

Patrón de diseño a utilizar:

Para este proyecto se implementará el patrón Repository, debido a que proporciona una capa de abstracción para el acceso a los datos. Cada entidad, tendrá su propio repositorio encargado de gestionar las operaciones sobre la base de datos. Este patrón favorece la aplicación del principio de responsabilidad única (SRP), mejora la organización del código, facilita el mantenimiento y permite cambiar la tecnología de persistencia sin afectar la lógica de negocio.

Infraestructura del proyecto:

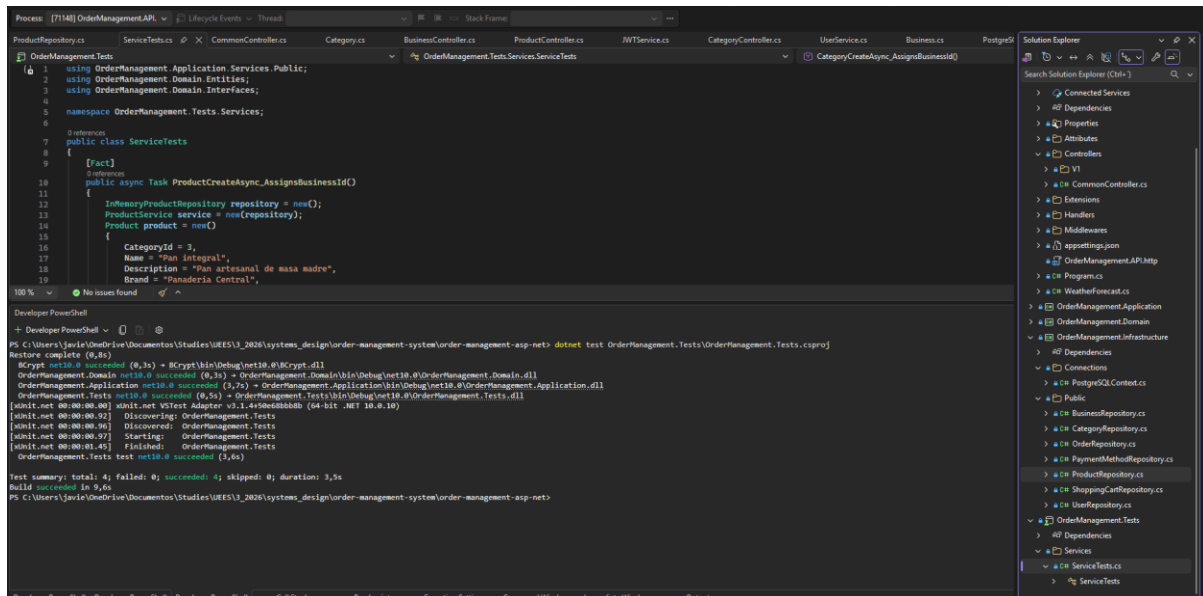
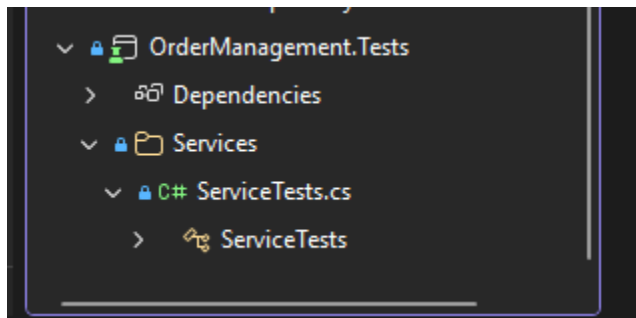


Código base + catálogo de malos olores:

2. Código base + informe de diagnóstico

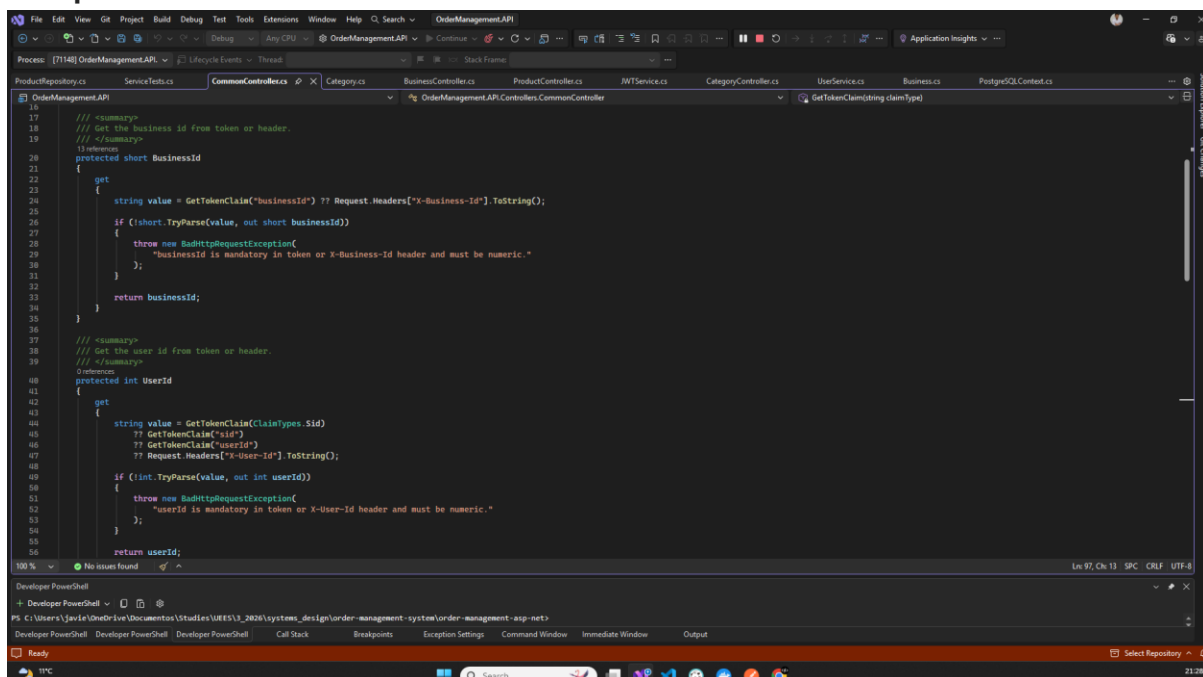
Enlace al **repositorio Git**: <https://github.com/Jonnathan1288/order-management-system.git>

Suite inicial de **pruebas unitarias** ejecutándose en verde.



Informe con **mínimo 5 malos olores** identificados en su propio código: captura, ubicación (archivo/línea) y justificación de por qué es un problema.

1. Captura:

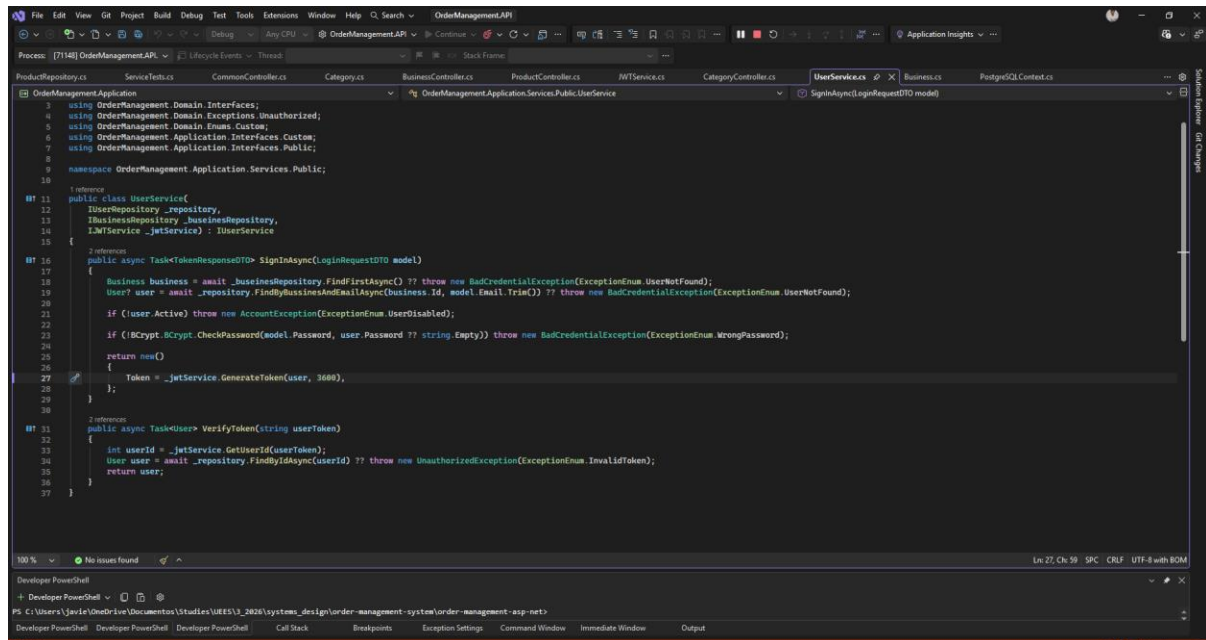


Ubicación:

OrderManagement.API/Controllers/CommonController.cs (líneas 20-78)

Mal olor identificado: Código duplicado**Justificación:**

Las propiedades BusinessId, UserId y CustomerId repiten la misma lógica: leer un claim, buscar un header, convertir el valor y lanzar excepción si no es numérico. Esto hace que el controlador base crezca innecesariamente y que cualquier cambio en la forma de leer headers o tokens deba repetirse varias veces.

2. Captura:

```
1 using OrderManagement.Domain.Interfaces;
2 using OrderManagement.Domain.Exceptions.Unauthorized;
3 using OrderManagement.Domain.Enums.Custom;
4 using OrderManagement.Application.Interfaces.Custom;
5 using OrderManagement.Application.Interfaces.Public;
6
7 namespace OrderManagement.Application.Services.Public;
8
9 1 reference
10 public class UserService(
11     IUserRepository _userRepository,
12     IBusinessRepository _businessRepository,
13     IJWTService _jwtService) : IUserService
14 {
15     2 references
16     public async Task<TokenResponseDTO> SignInAsync(LoginRequestDTO model)
17     {
18         Business business = await _businessRepository.FindFirstAsync() ?? throw new BadCredentialException(ExceptionEnum.UserNotFound);
19         User? user = await _userRepository.FindByBusinessIdAndEmailAsync(business.Id, model.Email.Trim()) ?? throw new BadCredentialException(ExceptionEnum.UserNotFound);
20         if (!user.Active) throw new AccountException(ExceptionEnum.UserDisabled);
21         if (!BCrypt.BCrypt.CheckPassword(model.Password, user.Password ?? string.Empty)) throw new BadCredentialException(ExceptionEnum.WrongPassword);
22         return new()
23         {
24             Token = _jwtService.GenerateToken(user, 3600),
25         };
26     }
27
28     2 references
29     public async Task<User> VerifyToken(string userToken)
30     {
31         int userId = _jwtService.GetUserId(userToken);
32         User user = await _userRepository.FindByIdAsync(userId) ?? throw new UnauthorizedException(ExceptionEnum.InvalidToken);
33         return user;
34     }
35 }
```

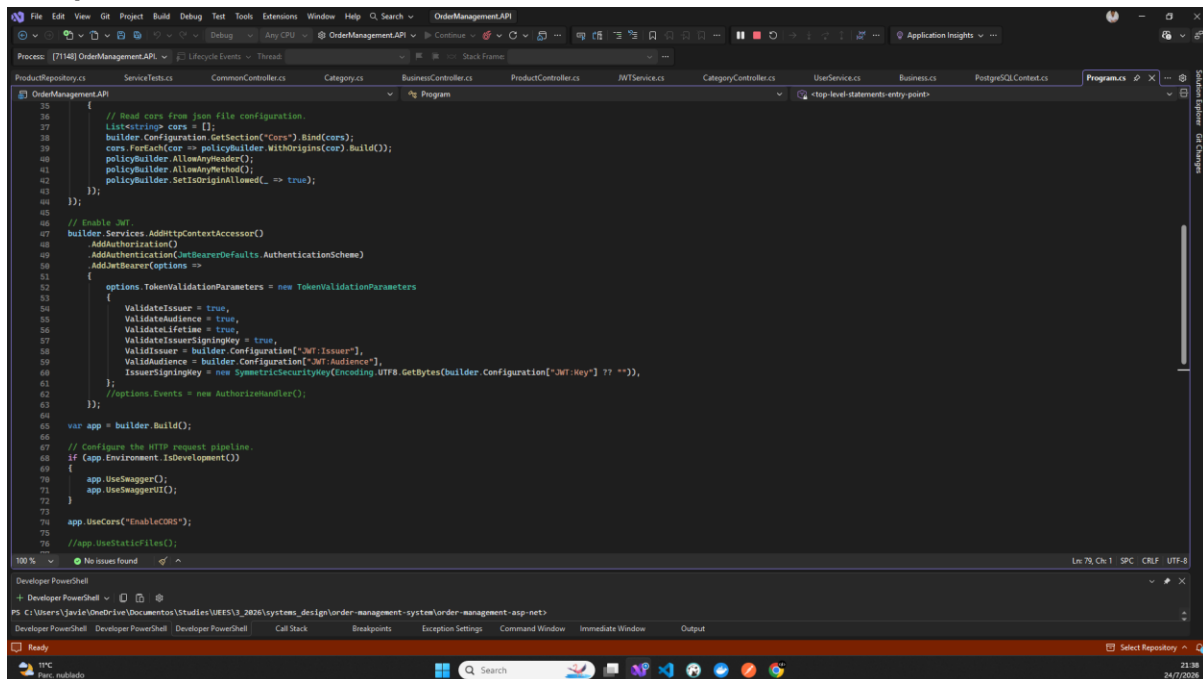
Ubicación:

OrderManagement.Application/Services/Public/UserService.cs (línea 27)

Mal olor identificado: Valor quemado**Justificación:**

El tiempo de vida del token está escrito directamente como 3600. Esto reduce la claridad del código y dificulta cambiar la duración del token en el futuro.

3. Captura:



```

35 {
36     // Read cors from json file configuration.
37     List<string> cors = [];
38     builder.Configuration.GetSection("Cors").Bind(cors);
39     cors.ForEach(cor => policyBuilder.WithOrigins(cor).Build());
40     policyBuilder.AllowAnyHeader();
41     policyBuilder.AllowAnyMethod();
42     policyBuilder.SetIsOriginAllowed(_ => true);
43 });
44 });
45
46 // Enable JWT
47 builder.Services.AddHttpContextAccessor()
48     .AddAuthorization()
49     .AddAuthentication(CertificateDefaults.AuthenticationScheme)
50     .AddJwtBearer(options =>
51     {
52         options.TokenValidationParameters = new TokenValidationParameters
53         {
54             ValidateIssuer = true,
55             ValidateAudience = true,
56             ValidateLifetime = true,
57             ValidateIssuerSigningKey = true,
58             IssuerSigningKey = builder.Configuration["JWT:Issuer"],
59             ValidAudience = builder.Configuration["JWT:Audience"],
60             IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["JWT:Key"] ?? ""));
61         };
62         options.Events = new AuthorizationEvents();
63     });
64
65 var app = builder.Build();
66
67 // Configure the HTTP request pipeline.
68 if (app.Environment.IsDevelopment())
69 {
70     app.UseSwagger();
71     app.UseSwaggerUI();
72 }
73
74 app.UseCors("EnableCORS");
75
76 //app.UseStaticFiles();
    
```

Ubicación:

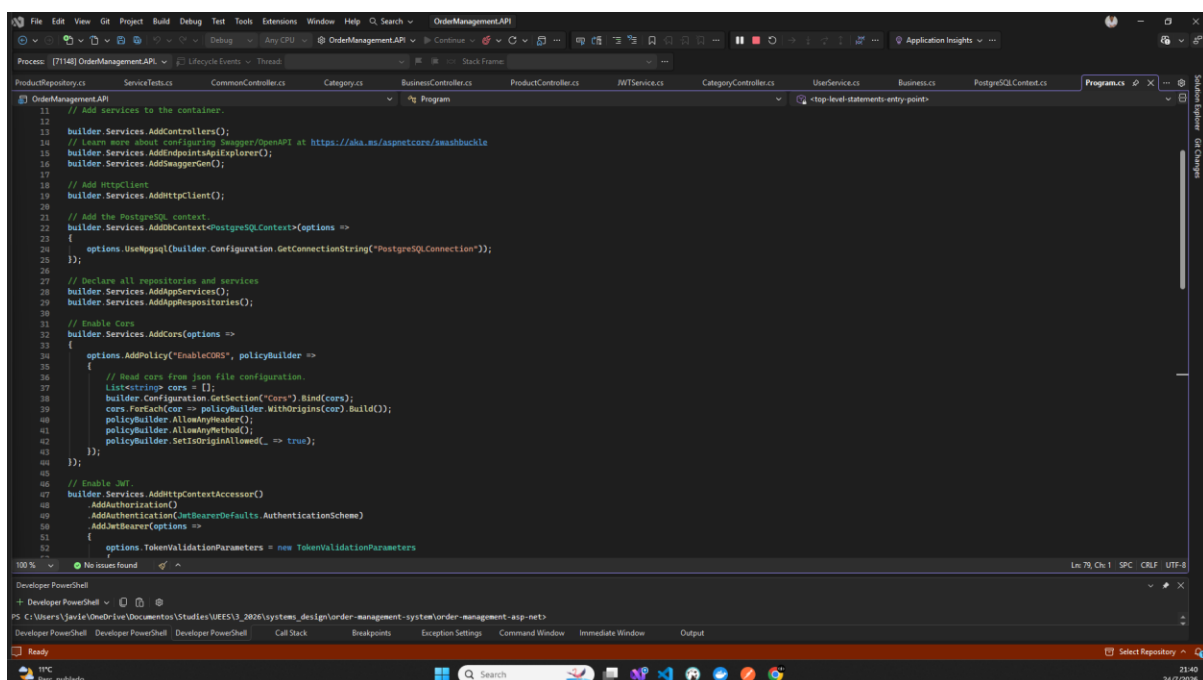
OrderManagement.API/Program.cs (líneas 47-63)

Mal olor identificado: Configuración JWT directamente en Program.cs

Justificación:

Toda la configuración del JWT está escrita dentro de Program.cs. Esto hace que el archivo crezca y mezcle responsabilidades de arranque, autenticación y configuración de seguridad. Podría separarse en un método de extensión.

4. Captura:



```

11 // Add services to the container.
12 builder.Services.AddControllers();
13 // Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
14 builder.Services.AddEndpointsApiExplorer();
15 builder.Services.AddSwaggerGen();
16
17 // Add HttpClient
18 builder.Services.AddHttpClient();
19
20 // Add the PostgreSQL context
21 builder.Services.AddDbContext<PostgreSQLContext>(options =>
22 {
23     options.UseNpgsql(builder.Configuration.GetConnectionString("PostgreSQLConnection"));
24 });
25
26 // Declare all repositories and services
27 builder.Services.AddAppServices();
28 builder.Services.AddAppRepositories();
29
30 // Enable CORS
31 builder.Services.AddCors(options =>
32 {
33     options.AddPolicy("EnableCORS", policyBuilder =>
34     {
35         // Read cors from json file configuration.
36         List<string> cors = [];
37         builder.Configuration.GetSection("Cors").Bind(cors);
38         cors.ForEach(cor => policyBuilder.WithOrigins(cor).Build());
39         policyBuilder.AllowAnyHeader();
40         policyBuilder.AllowAnyMethod();
41         policyBuilder.SetIsOriginAllowed(_ => true);
42     });
43 });
44
45 // Enable JWT
46 builder.Services.AddHttpContextAccessor()
47     .AddAuthorization()
48     .AddAuthentication(CertificateDefaults.AuthenticationScheme)
49     .AddJwtBearer(options =>
50     {
51         options.TokenValidationParameters = new TokenValidationParameters
52         {
53             ValidateIssuer = true,
54             ValidateAudience = true,
55             ValidateLifetime = true,
56             ValidateIssuerSigningKey = true,
57             IssuerSigningKey = builder.Configuration["JWT:Issuer"],
58             ValidAudience = builder.Configuration["JWT:Audience"],
59             IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["JWT:Key"] ?? ""));
60         };
61         options.Events = new AuthorizationEvents();
62     });
63
64 var app = builder.Build();
    
```

Ubicación:

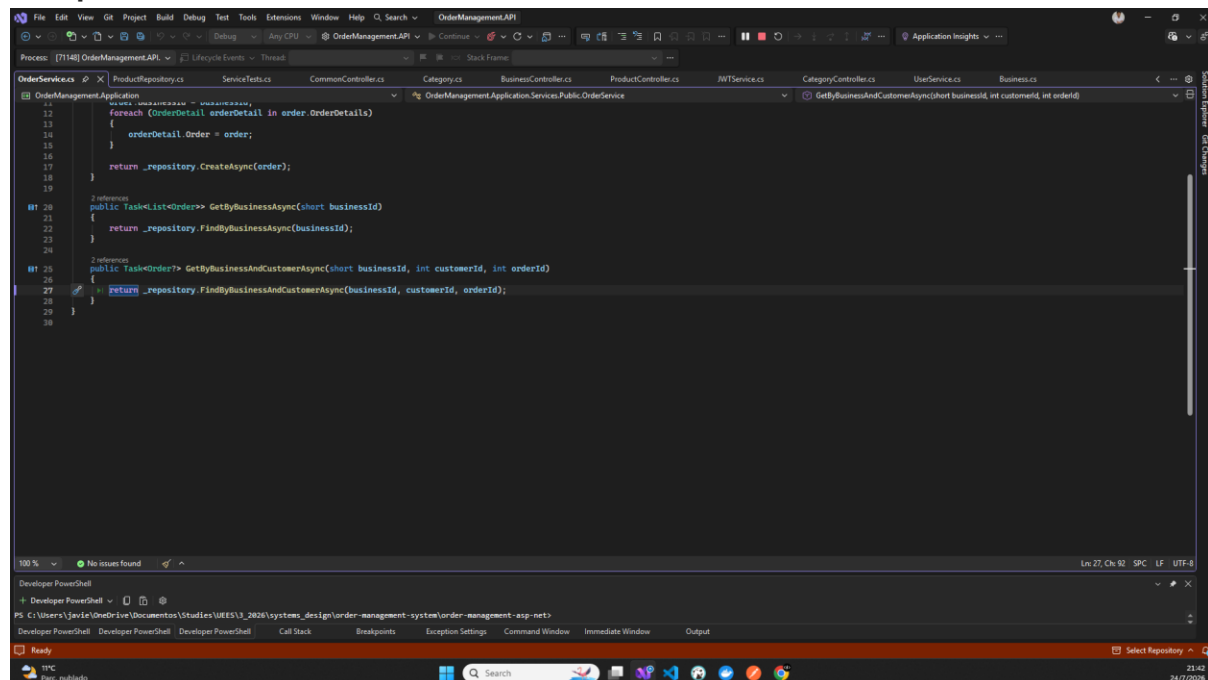
OrderManagement.API/Program.cs (líneas 27-29)

Mal olor identificado: Error tipográfico en nombre de método

Justificación:

El método se llama AddAppRespositories, cuando debería ser AddAppRepositories. Aunque compila, el nombre mal escrito reduce la claridad y da mala impresión de mantenimiento.

5. Captura:



Ubicación:

OrderManagement.Application/Services/Public/OrderService.cs (líneas 20-23)

Mal olor identificado: Retorno nulo sin manejo claro

Justificación:

El método puede devolver Order?, pero no define qué debe pasar si no encuentra el pedido. Esto puede terminar en respuestas con data: null en vez de un 404 Not Found, haciendo menos clara la API.

FASE 3

Mínimo **6 refactorizaciones** que cubran los 4 niveles: métodos, clases/objetos, datos y condicionales

Un commit por cada refactorización (esto es clave: te permite ver el diff antes/después y verificar que las pruebas pasaban en ambos puntos; un solo commit gigante no es válido)

1 - OrderService:

Se refactorizó OrderService extrayendo la lógica de preparación de una orden a un método privado. Antes, CreateAsync asignaba el BusinessId, registraba los detalles y creaba la orden

en el mismo bloque. Ahora CreateAsync queda más claro, delegando la preparación a PrepareOrderForCreation, lo que mejora la legibilidad y separa responsabilidades dentro del servicio.

SHA: 50f756d4568b1b3a3d54d160b150104998aa5243

Commit: <https://github.com/Jonnathan1288/order-management-system/commit/50f756d4568b1b3a3d54d160b150104998aa5243>

2 – CommonController:

Se refactorizó CommonController para eliminar código repetido en la lectura de identificadores. Las propiedades BusinessId, UserId y CustomerId ahora usan métodos auxiliares que centralizan la lectura desde token o headers y la conversión numérica. Esto mejora la legibilidad y facilita mantener la lógica de autenticación contextual en un solo lugar.

SHA: 5aec65db775b92685821d25125caea1e94f97044

Commit: <https://github.com/Jonnathan1288/order-management-system/commit/5aec65db775b92685821d25125caea1e94f97044>

3 – JwtAuthenticationExtension + Program

Se refactorizó la configuración JWT extrayéndola de Program.cs hacia un método de extensión llamado AddJwtAuthentication. Antes, Program.cs mezclaba configuración general de la aplicación con detalles de autenticación y validación del token. Ahora esa responsabilidad queda separada en una clase especializada, mejorando la organización, legibilidad y mantenibilidad del arranque de la aplicación.

SHA: 7a5e2821b369542fd8bfc0fe063921b3c86162f3

Commit: <https://github.com/Jonnathan1288/order-management-system/commit/7a5e2821b369542fd8bfc0fe063921b3c86162f3>

4 – SwaggerExtension + Program

Se refactorizó la configuración de Swagger extrayendo el condicional de entorno desde Program.cs hacia un método de extensión llamado UseSwaggerDocumentation. Antes, el archivo principal contenía directamente la lógica para habilitar Swagger solo en desarrollo. Ahora esa condición queda encapsulada, haciendo que Program.cs sea más limpio y fácil de leer.

SHA: 6a72269f320c71551e1f058d000a794691a636d5

Commit: <https://github.com/Jonnathan1288/order-management-system/commit/6a72269f320c71551e1f058d000a794691a636d5>

5 – Diagrama de clases + Emun de estados

Se eliminó el enum de estados que no estaba siendo usado en el código y se actualizó el diagrama de clases para representar el estado de Pedido y CarritoCompras como string, alineándolo con las entidades reales del backend.

SHA: 40134e1a207a45c57721d6b7254cf8cec458d913

Commit: <https://github.com/Jonnathan1288/order-management-system/commit/40134e1a207a45c57721d6b7254cf8cec458d913>

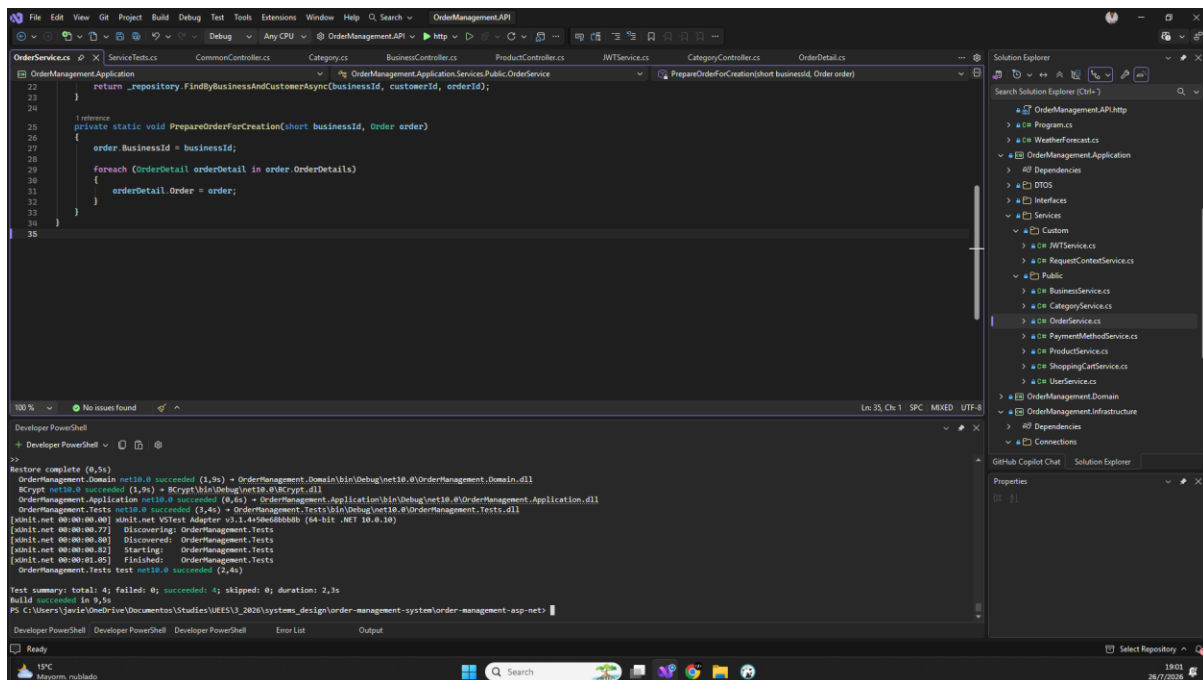
6 – CommonController + RequestHeaders

Se centralizaron los nombres de headers usados para obtener BusinessId, UserId y CustomerId. Antes de los valores "X-Business-Id", "X-User-Id" y "X-Customer-Id" estaban escritos directamente en CommonController; ahora están definidos en la clase RequestHeaders, reduciendo cadenas repetidas y posibles errores de escritura.

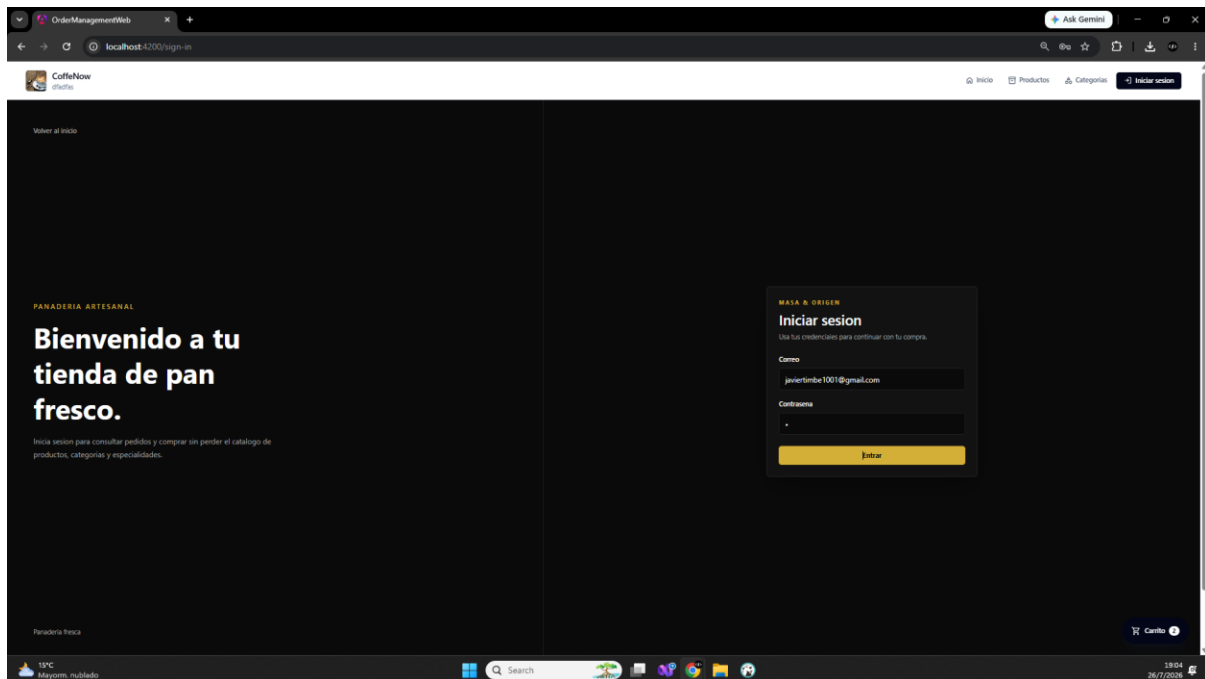
SHA: a26df1a678b20935da4e5b2b25d347b5eea9767e

Commit: <https://github.com/Jonnathan1288/order-management-system/commit/a26df1a678b20935da4e5b2b25d347b5eea9767e>

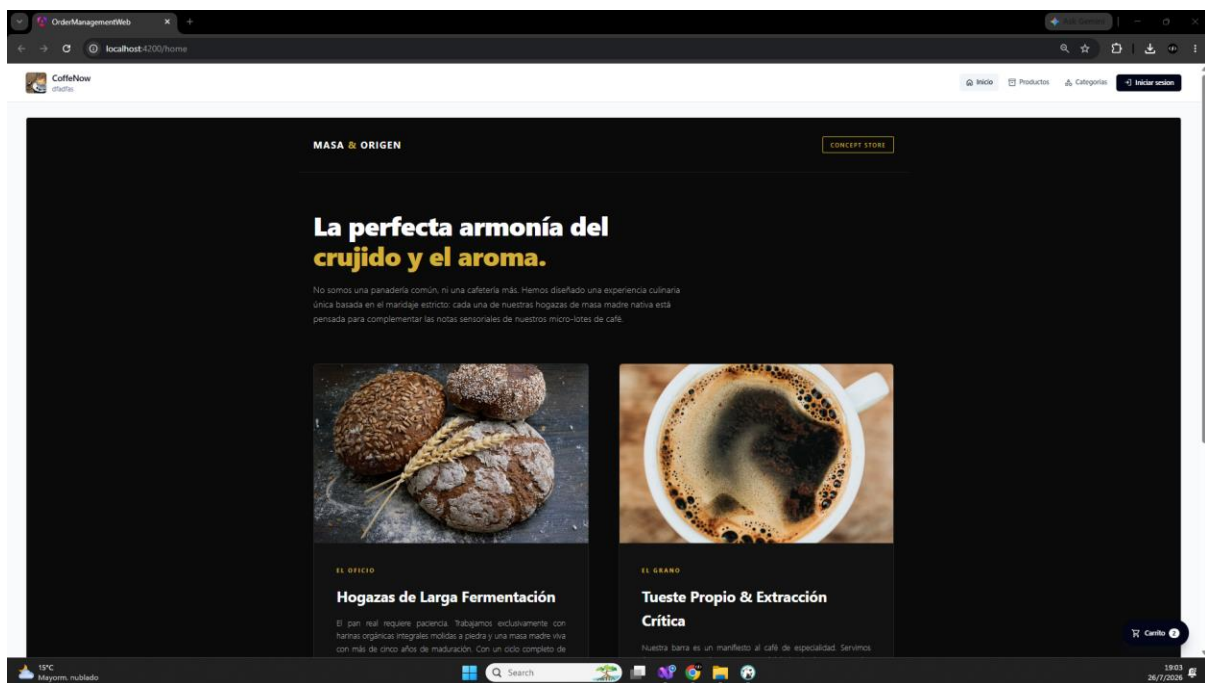
Test Funcional



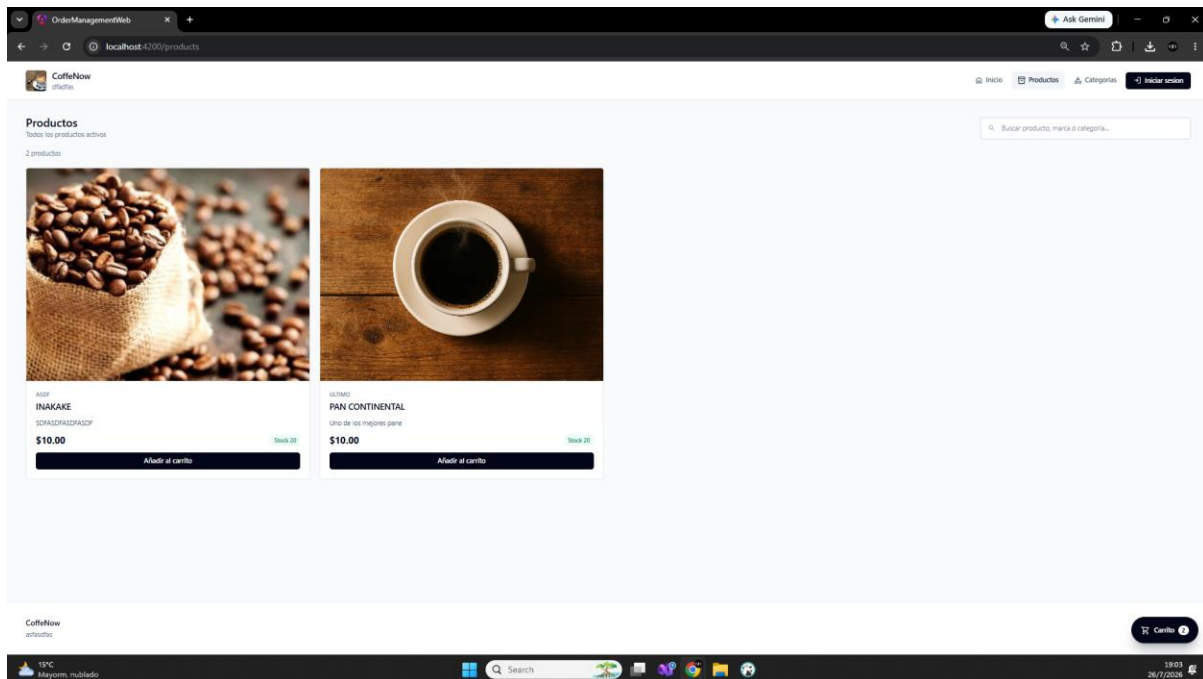
Login:



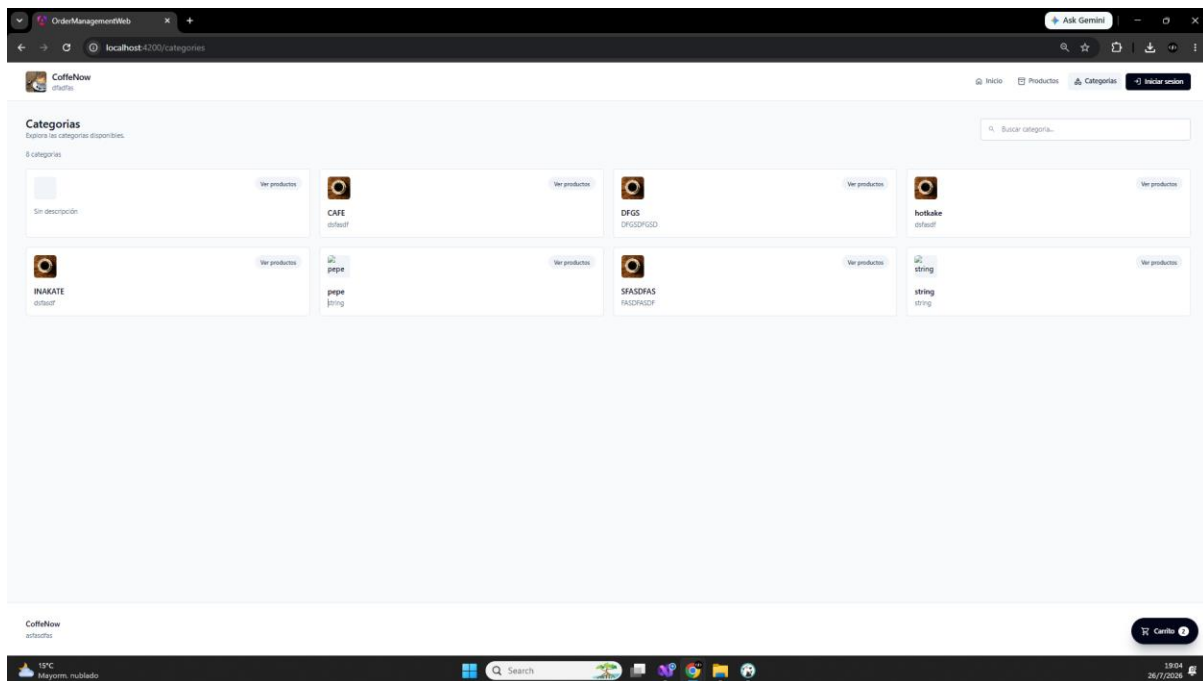
Pantalla inicial:



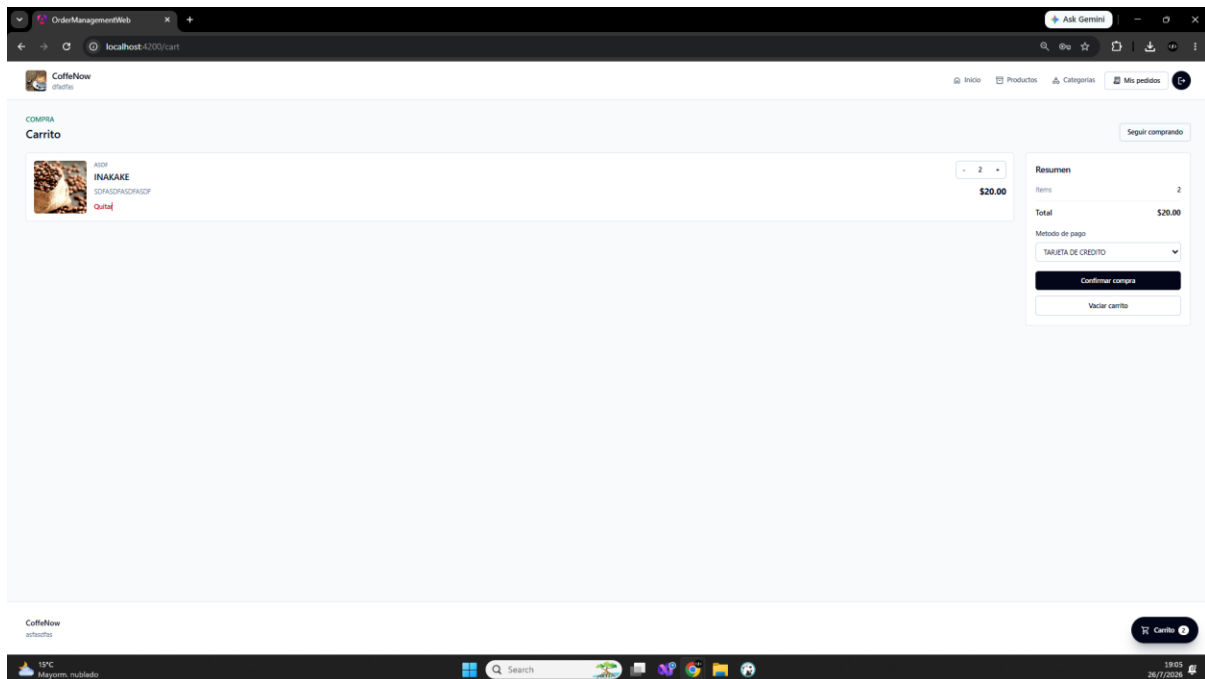
Productos:



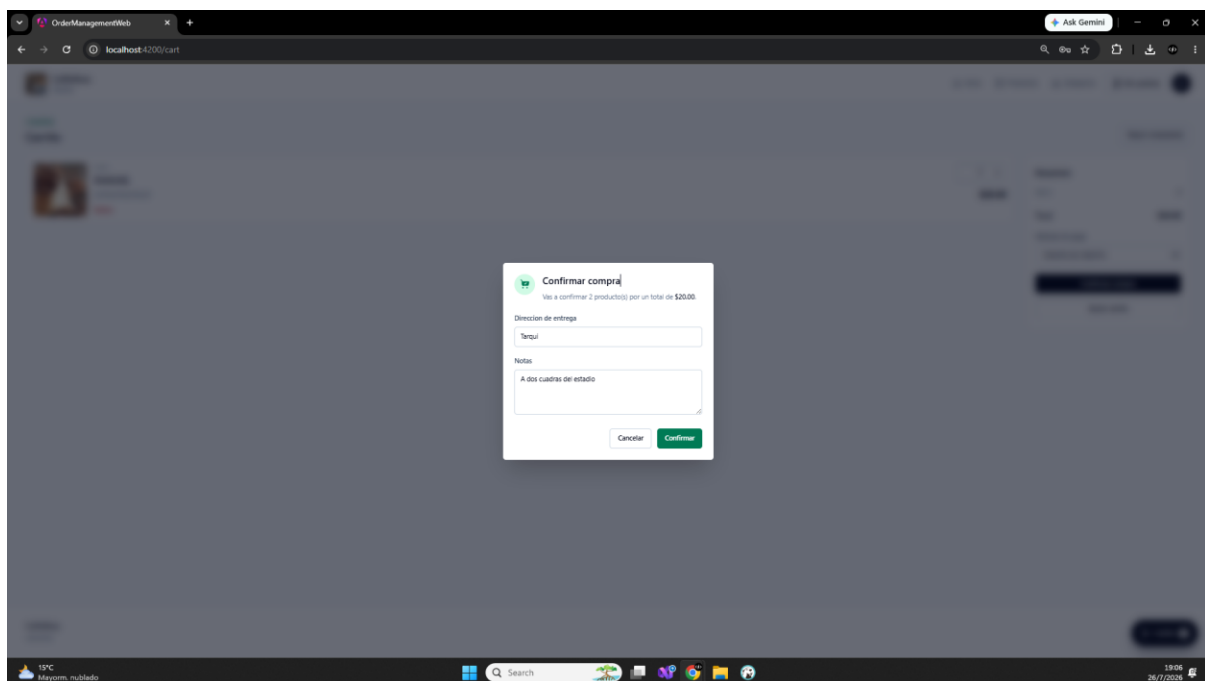
Categorías:



Carrito de compras:



Confirmar pedido:



Pedido ejecutado:

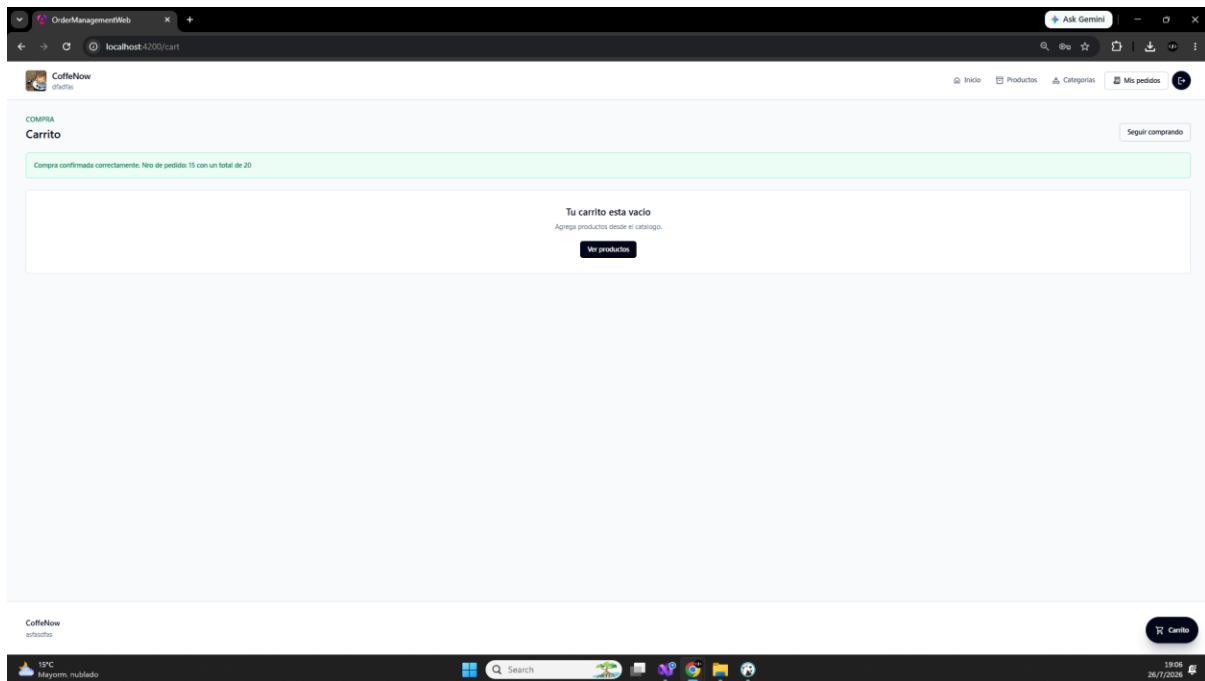


Diagrama de clases actualizado mostrando cómo evolucionó el diseño

Sistema de Gestión de Pedidos - Diagrama de Clases

